

Praktikum „Bildverarbeitung“ (WS 2025/26 – Dr.-Ing. Lorenz Fieten) – 1. Aufgabe

Ein punktbasiertes Verfahren, welches z. B. zur Bildverbesserung bei der Darstellung von medizinischen CT-Daten eingesetzt wird, ist die sog. Fensterung (engl.: „windowing“). Dabei wird der Kontrast in einem vorgegebenen Grauwertbereich auf Kosten des Kontrasts in den anderen Grauwertbereichen erhöht.

Gegenstand der Praktikumsaufgabe ist die Entwicklung einer kleinen auf der OpenCV-Infrastruktur basierenden Anwendung, in der der Benutzer interaktiv den relevanten Grauwertbereich definiert.

Primäre Lernziele sind die Erkennung der Relevanz von Bildverbesserung sowie der Umgang mit OpenCV-Datenstrukturen, OpenCV-Basis-Funktionalitäten (z. B. Laden/ Speichern/ Anzeigen und Manipulieren von Bildern, rudimentäre Benutzer-Interaktion) und OpenCV-Online-Hilfe-Seiten.

1. Öffnen Sie mittels ImageJ (File -> Open Sample...) die Datei m51.tif („M51 Galaxy“).
2. Speichern Sie diese Datei unter demselben Namen an einem Ort Ihrer Wahl ab.
3. Legen Sie ein neues Projekt an (Konsolen-App): z. B. „ConsoleApplication_Windowing“.
4. Erstellen Sie ein lokales Git-Repository.
5. Setzen Sie die Projekt-Eigenschaften so, dass OpenCV benutzt werden kann.
 - a. Für alle Konfigurationen unter C/C++ -> Allgemein
„Zusätzliche Includeverzeichnisse“: C:\opencv\build\include
 - b. Für alle Konfigurationen unter Linker -> Allgemein
„Zusätzliche Bibliotheksverzeichnisse“: C:\opencv\build\x64\vc16\lib
 - c. Für Debug-Konfiguration unter Linker -> Eingabe
„Zusätzliche Abhängigkeiten“: opencv_world4120d.lib
 - d. Für Release-Konfiguration unter Linker -> Eingabe
„Zusätzliche Abhängigkeiten“: opencv_world4120.lib eintragen
6. Fügen Sie oberhalb von `int main()` `#include „opencv2/opencv.hpp“` ein.
7. Legen Sie innerhalb von `int main()` ein Objekt vom Typ `cv::Mat` an: z. B. `inputImage`.
8. Sobald sich der Code kompilieren lässt, committen Sie Ihren Code.
9. Laden Sie in `int main()` die Datei m51.tif.
Zur Orientierung: <https://opencv.org/blog/read-display-and-write-an-image-using-opencv/>
10. Analysieren Sie das Bild mittels Image Watch:
 - a. Welche Eigenschaften hat die `cv::Mat`?
 - b. Welchen Bildinhalt kann man erkennen?
 - c. Welchen Bildinhalt kann man nach einer Betrachtung der Grauwerte vermuten?
 - d. Wie lässt sich die Darstellung ggf. verbessern?

11. Ermitteln Sie den in der vorliegenden `cv::Mat` maximal darstellbaren Wert `upperLimit`.
12. Geben Sie den Wert von `upperLimit` mit dem Format von `inputImage` auf der Konsole aus.
13. Sobald sich der Code kompilieren lässt, committen Sie Ihren Code.
14. Zeigen Sie das Bild in einem Fenster mit dem Namen "Input image" an.
Zur Orientierung: <https://opencv.org/blog/read-display-and-write-an-image-using-opencv/>
15. Unterdrücken Sie ggf. störende Log-Meldungen auf der Konsole.
Hinweis: Werfen Sie einen Blick in den Namespace `cv::utils::logging`.
16. Sobald sich der Code kompilieren lässt, committen Sie Ihren Code.
17. Deklarieren Sie 2 Schwellwerte `int g_upperThreshold, g_lowerThreshold = 0` als globale Variablen.
18. Setzen Sie den oberen Schwellwert auf ca. $1/16$ des maximal darstellbaren Werts `upperLimit`.
19. Erzeugen Sie ein 8-Bit-Output-Bild und berechnen seine Grauwerte wie folgt:
 - a. Pixel, die im Input-Bild Werte unterhalb des unteren Schwellwerts annehmen: 0
 - b. Pixel, die im Input-Bild Werte oberhalb des oberen Schwellwerts annehmen: 255
 - c. Restliche Pixel: Werte durch lineare Interpolation (und Rundung)
20. Zeigen Sie das Output-Bild in einem Fenster mit dem Namen "Output image" an.
21. Vergleichen Sie das Output-Bild mit dem Input-Bild:
 - a. Was kann auf dem Input-Bild besser erkannt werden?
 - b. Was kann auf dem Output-Bild besser erkannt werden?
22. Sobald sich der Code kompilieren lässt, committen Sie Ihren Code.
23. Fügen Sie in `int main()` an geeigneter Stelle die folgenden Code-Zeilen ein:
 - a. `cv::namedWindow("Output image", cv::WINDOW_AUTOSIZE);`
 - b. `cv::createTrackbar("Lower threshold", "Output image", &g_lowerThreshold, upperLimit);`
 - c. `cv::createTrackbar("Upper threshold", "Output image", &g_upperThreshold, upperLimit);`
24. Nutzen Sie diese Code-Zeilen zur interaktiven Variation der Schwellwerte.
Hinweis: Mit `cv::waitKey(10) == 27` können Sie das Drücken der Escape-Taste feststellen.
25. Sobald sich der Code kompilieren lässt, committen Sie Ihren Code.
26. Ergänzen Sie zur Ausgabe von Schwellwert-Änderungen auf der Konsole diese Funktionen:
 - a. `static void onLowerThresholdTrackbar(int pos, void*)`
 - b. `static void onUpperThresholdTrackbar(int pos, void*)`

27. Verknüpfen Sie die neue Funktionalität mit Ihrem bisherigen Code wie folgt:
 - a. `cv::createTrackbar("Lower threshold", "Output image", nullptr, upperLimit, onLowerThresholdTrackbar);`
 - b. `cv::createTrackbar("Upper threshold", "Output image", nullptr, upperLimit, onUpperThresholdTrackbar);`
28. Fügen Sie an passender Stelle die folgenden Code-Zeilen ein:
 - a. `cv::setTrackbarPos("Lower threshold", "Output image", g_lowerThreshold);`
 - b. `cv::setTrackbarPos("Upper threshold", "Output image", g_upperThreshold);`
29. Sobald sich der Code kompilieren lässt, committen Sie Ihren Code.
30. Überlegen Sie sich, warum bestimmte Schwellwert-Kombinationen auszuschließen sind.
31. Passen Sie den Code so an, dass diese Schwellwert-Kombinationen ausgeschlossen sind.
32. Sobald sich der Code kompilieren lässt, committen Sie Ihren Code.
33. Ergänzen Sie einen Modus, in dem Pixel in der Sättigung farbig dargestellt werden:
 - a. Dunkle Pixel in der Sättigung sollen blau dargestellt werden.
 - b. Helle Pixel in der Sättigung sollen rot dargestellt werden.
34. Ermöglichen Sie einen Wechsel zwischen den Modi durch Drücken der Taste ‚m‘.
35. Sobald sich der Code kompilieren lässt, committen Sie Ihren Code.
36. Ermöglichen Sie ein Abspeichern des Output-Bilds durch Drücken der Taste ‚s‘:
 - a. Das Output-Bild soll als PNG-Bild abgespeichert werden.
 - b. Der Dateiname des Output-Bilds soll sich an dem des Input-Bilds orientieren.
 - c. Der Dateiname des Output-Bilds soll die Schwellwerte beinhalten.
37. Sobald sich der Code kompilieren lässt, committen Sie Ihren Code.
38. Öffnen Sie mittels ImageJ (File -> Open Sample...) die Datei ct.dcm.tif („CT“).
39. Speichern Sie diese Datei unter demselben Namen an einem Ort Ihrer Wahl ab.
40. Laden Sie in `int main()` die Datei ct.dcm.tif wie zuvor die Datei m51.tif.
41. Betrachten Sie das Bild mit verschiedenen Schwellwerten:
 - a. Mit welchen Schwellwerten lassen sich welche Bereiche gut darstellen?
 - b. Welche Anwendungen könnten besonders von der Bildverbesserung profitieren?
42. Recherchieren Sie, für welche Unteraufgaben optimierte OpenCV-Funktionalität existiert.
43. Testen Sie, inwiefern sich damit ein Performance-Gewinn feststellen lässt.
44. Überlegen Sie, wie die entwickelte Funktionalität sinnvoll erweitert werden könnte.